

V2 Feature Engineering Process

Selected candidate: v2_xgboost_safe_plus_rolling ใช้ XGBoost และ feature 60 ตัวแบบ order-time safe

Input / Output

Item	Path / Value
Input clean data	data\processed\clean_dataset.csv
Engineered dataset	docs\version 2\data\features\df_engineered_v2_preview.csv
Used feature count	60
Code files	docs\version 2\feature_engineering_v2.py docs\version 2\train_v2_optimized_model.py docs\version 2\v2_xgboost_safe_plus_rolling\scripts\train_v2_optimized_model.py

Feature Engineering Steps

1. เริ่มจาก clean_dataset.csv แล้วลด feature ให้ดีดความง่ายกว่า V1
2. สร้าง customer_tenure_months, order_month, order_dayofweek, is_weekend, age_group และ logistics_risk
3. ตัด post-delivery fields สำหรับ selected candidate เช่น delivery_days และ delay_days
4. เพิ่ม rolling customer history 30/60/90/180/365 วัน เช่น hist_return_rate_30d และ hist_spend_sum_365d
5. เพิ่ม interaction features เช่น discount_amount_ratio, category_payment, category_channel, province_payment
6. ใช้ target encoding/smoothing สำหรับ categorical feature
7. Train XGBoost และเลือก threshold 0.49 จาก balanced metric

Feature Groups Created / Used

Feature group	Count
customer_history	23
promotion_discount	8
customer_profile	7
logistics	6
product_category	5
time	4
payment_channel	3
price_amount	3
other	1

Reasoning

V2 selected ถูกเลือกเพราะใช้ข้อมูลจริง, order-time safe, accuracy สูงสุดในกลุ่มข้อมูลจริง, precision ดีขึ้น, feature สอดคล้องกับ Feature Store และไม่มี delivery_days/delay_days leakage

Model Result Snapshot

Metric	Value
Model	XGBClassifier
Accuracy	71.07%
Recall	56.88%
Precision	50.20%
F1	53.33%

Metric	Value
AUC	71.47%
Cost	20,250
Rating	B

Pseudo-code / Code Logic

```
df = read_csv("clean_dataset.csv")
df = add_v2_basic_features(df)
df = add_customer_tenure_order_time_age_group(df)
df = add_logistics_risk(df)
df = drop_post_event_for_safe_candidate(["delivery_days", "delay_days"])
df = add_rolling_history_windows([30, 60, 90, 180, 365])
df = add_interactions(["category_payment", "category_channel", "province_payment"])
X = target_encode_with_smoothing(df[selected_features])
model = XGBClassifier(...)
model.fit(X_train, y_train)
```